CSC 2210 — Object Oriented Programming 2

Project Assignment 01: System Design & Report Submission

American International University–Bangladesh (AIUB)

Faculty of Science and Technology · Department of Computer Science

Semester: Summer 2025-2026

Group No: 05

Section: AA

Project Name: Grocery / Super Shop System Design

Supervisor: Dr. Md. Iftekharul Mobin

NAME	ID
SAHIL SUBAIR	25-62082-2
NAFIM ISLAM NABID	24-58032-2
ESRAT JAHAN AKHI	23-55468-3
REDWANUL ALLAM SEAM	23-51158-1

CO2 — Display and verify the meaning of a real-life project using C# GUI concepts with database integration — 15 marks

Criterion	Marks	What earns "Excellent (5)" in this assignment
Requirement fulfillment		Case study describes a genuine, detailed real-life scenario. All three roles are meaningful in that domain. Every mandatory feature from Section 1 is present in the requirements and traceable to a form in the navigation diagram.
Validation		Every user story states what is validated and how the system responds. At least 2 mockups visibly show validation/error states. Database constraints (NOT NULL, UNIQUE, CHECK) enforce the same rules.
Verification		The data flow is proven end-to-end: the navigation diagram, the schema and the SQL queries agree with each other. Every feature has a query behind it; every query touches a table that exists in the schema.

CO3 — Prepare and explain a real-life desktop application synthesizing C# components and development tools — 15 marks

Criterion	Marks	What earns "Excellent (5)" in this assignment
Organization of the application		Repo follows the required folder structure. README is complete, well-ordered and readable. PDF report has cover page and all 8 chapters. All members have commits. Work distribution table is honest and specific.
Representation and Integration of Database		6+ tables, correctly normalized to 3NF with justification. All PK/FK relationships drawn and labelled. Junction table present. SQL script runs without error and uses JOIN, GROUP BY, HAVING and aggregates.
Graphical User Interface		12+ form designs, visually consistent, user-friendly and coherent. Navigation diagram is complete with labelled arrows and the role-decision branch. Every form is reachable and every form has a way back.

Bonus

Item	Marks
Correct ER Diagram (entities, attributes, relationships, cardinality)	
Working login + role routing already implemented in C#	

TABLE OF CONTENTS

Chapter 1 — Introduction & Case Study

1.1 Introduction

Running a modern grocery supermarket or an online grocery marketplace isn't just about stocking shelves—it's about juggling a million moving parts behind the scenes. From independent shop owners trying to keep track of daily stock to platform administrators managing commissions and store approvals, things can get messy very quickly. That is exactly why we built Grocery Super_Shop. It's a desktop application crafted in C# and Windows Forms (WinForms) designed to bring order to the chaos. By establishing clear boundaries for Super Admins, Shop Admins, and everyday Customers, this system handles everything from inventory tracking to automated revenue splitting under one roof.

1.2 Case Study

Think about how traditional multi-vendor platforms operate. Usually, store owners complain about opaque commission cuts, while platform managers struggle with decentralized inventory updates and unmoderated customer reviews. Grocery-Super_Shop solves these exact pain points by acting as a single, cohesive hub where multiple vendors can operate independently under a single master network.

Chapter 2 — Functional Requirements & User Stories

2.1 Functional Requirements

The system's functional requirements are categorized by user roles to ensure clear operational boundaries and granular access control:

Super Admin Requirements:o

Shop Management: The system must allow the Super Admin to add, remove, or Search independent shop vendor
accounts.

Platform Sales & Commission: The system must calculate and display global platform revenue, order volumes, and automated commission percentages across all registered shops.

Shop Admin Requirements:

Product Management: The system must allow Shop Admins to add new catalog items, update pricing and descriptions, archive inactive products, and perform bulk imports.

Inventory Control: The system must track stock quantities in real time, generate low-stock alerts, and allow inventory reorder thresholds to be set.

Sales & Earnings: The system must display daily and weekly gross sales, net earnings charts, average order values, and detailed transaction logs.

Offers & Promotions: The system must enable Shop Admins to create, edit, and deactivate discount codes, flash sales, and promotional bundles.

Customer Requirements:

Product Browsing: The system must allow customers to search products by keyword or SKU, and filter items by category or price.

Cart & Checkout: The system must support shopping cart management, promo voucher application, and secure order placement.

2.2 User Stories

As a Super Admin, I want to view global platform commissions so that I can accurately track and audit overall platform revenue.

As a Super Admin, I want to moderate customer reviews across multiple stores so that platform quality and brand safety are maintained.

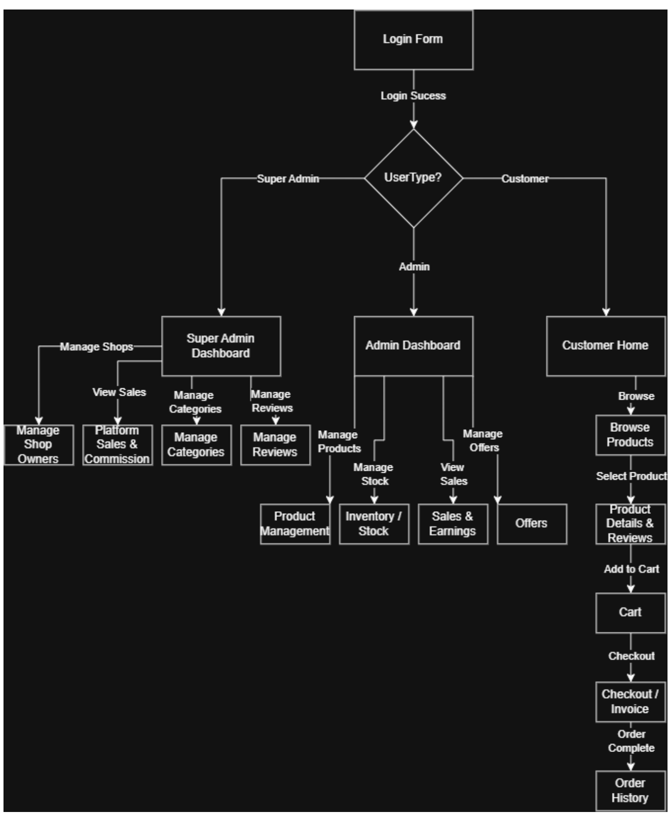
As a Shop Admin, I want to receive low-stock alerts so that I can reorder inventory before popular grocery items run out.

As a Shop Admin, I want to monitor gross sales and net earnings trends so that I can evaluate my shop's financial performance.

As a Customer, I want to filter products by category so that I can quickly find grocery items without endless scrolling.

As a Customer, I want to apply promotional coupon codes at checkout so that I can receive discounts on my grocery purchases.

Chapter 3 — UI Navigation Diagram



Everything kicks off at the Login Form, where users sign in and get instantly routed based on their specific role. If you are logged in as a Super Admin, your dashboard lets you handle shop owners, check platform-wide sales and commissions, organize categories, and moderate reviews all in one place. If you are an Admin managing an individual shop, your dashboard branches out into product management, stock levels, sales earnings, and promotional offers. Meanwhile, customers land on a straightforward storefront where they can browse items, check product details, manage their cart, breeze through checkout, and look back at their order history once everything is wrapped up.

Chapter 4 — Database Design

Sql Schema Diagram

ER DIAGRAM

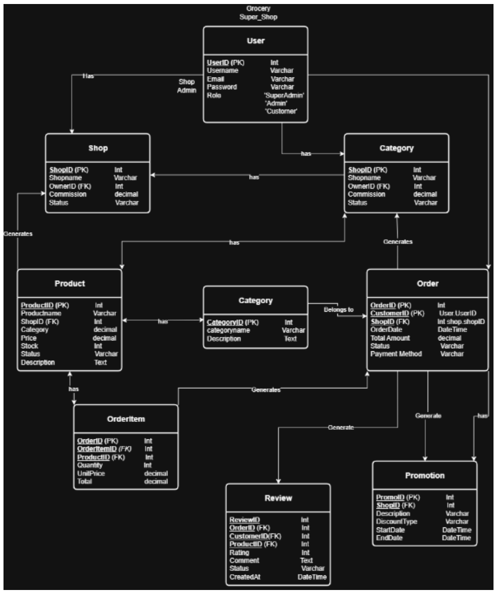


Table-by-Table Specifications

1. Users Table

Stores everyone who logs into the system, separating them by their specific operational roles.

- **UserID** (INT, Primary Key): The unique identification number for every registered account.
- **Username** (VARCHAR): The display name chosen by the user.
- **Email** (VARCHAR): The unique email address used for login and notifications.
- **PasswordHash** (VARCHAR): The securely encrypted password string.

- **Role** (VARCHAR): Defines permissions in the system (e.g., 'SuperAdmin', 'ShopAdmin', 'Customer').
- **CreatedAt** (DATETIME): Timestamp showing when the account was created.

2. Shops Table

Manages the independent vendor storefronts operating within our platform.

- **ShopID** (INT, Primary Key): The unique identifier for each grocery store.
- **ShopName** (VARCHAR): The commercial name of the vendor shop.
- **OwnerID** (INT, Foreign Key referencing Users.UserID): Links the shop directly to its designated admin.
- **CommissionRate** (DECIMAL): The specific percentage cut the platform takes from the shop's sales.
- **Status** (VARCHAR): Tracks the operational state (e.g., 'Active', 'Suspended').

3. Categories Table Organizes grocery items into clean, manageable departments.

- **CategoryID** (INT, Primary Key): The unique identifier for each product category.
- **CategoryName** (VARCHAR): The name of the department (e.g., 'Fruits', 'Dairy', 'Bakery').
- **Description** (TEXT): A brief summary of what types of items belong in this category.

4. Products Table

Holds the entire catalog of grocery items available across different vendor shops.

- **ProductID** (INT, Primary Key): The unique identifier for each individual product listing.
- **SKU** (VARCHAR): The stock-keeping unit code used for quick scanning and tracking.
- **ProductName** (VARCHAR): The title of the grocery item.
- **ShopID** (INT, Foreign Key referencing Shops.ShopID): Identifies which vendor owns this product.

•

CategoryID (INT, Foreign Key referencing Categories.CategoryID):

Groups the product into its respective department.

•

Price (DECIMAL): The cost per unit.

•

StockQty (INT): The current physical inventory count available.

•

LowStockThreshold (INT): The minimum stock level that triggers an

automatic low-stock warning.

•

Status (VARCHAR): Indicates whether the item is active or archived.

5. Orders Table

Tracks checkout transactions made by customers.•

OrderID (INT, Primary Key): The unique identifier for every processed

customer order.

•

CustomerID (INT, Foreign Key referencing Users.UserID): The customer

who placed the order.

•

ShopID (INT, Foreign Key referencing Shops.ShopID): The specific

vendor store fulfilling the order.

•

OrderDate (DATETIME): The exact timestamp when the order was submitted.

- **TotalAmount** (DECIMAL): The final calculated cost of the order.
- **Status** (VARCHAR): Current order progress (e.g., 'Pending', 'Completed', 'Cancelled').
- **PaymentMethod** (VARCHAR): The payment option selected by the customer.

6. OrderItem Table

Breaks down individual items included within a specific order.

- **OrderItemID** (INT, Primary Key): The unique identifier for the line item.
- **OrderID** (INT, Foreign Key referencing Orders.OrderID): Links the item back to its parent order.
- **ProductID** (INT, Foreign Key referencing Products.ProductID): Identifies the exact product purchased.
- **Quantity** (INT): How many units of this product were bought.

•

UnitPrice (DECIMAL): The price of a single unit at the time of purchase.

•

Subtotal (DECIMAL): The calculated total for this specific line item (Quantity * UnitPrice).

7. Review Table

Handles customer feedback and ratings submitted for products and stores.•

ReviewID (INT, Primary Key): The unique identifier for the review.

•

OrderID (INT, Foreign Key referencing Orders.OrderID): Links the review to a verified purchase.

•

CustomerID (INT, Foreign Key referencing Users.UserID): The user who wrote the review.

•

ProductID (INT, Foreign Key referencing Products.ProductID): The item being evaluated.

•

Rating (INT): The numerical score given by the customer.

•

Comment (TEXT): The written text feedback.

•

Status (VARCHAR): Moderation status (e.g., 'Pending', 'Approved', 'Flagged').

8. Promotion Table

Manages discount codes, flash sales, and special vendor offers.

- **PromoID** (INT, Primary Key): The unique identifier for the promotion.
- **ShopID** (INT, Foreign Key referencing Shops.ShopID): The shop offering the discount.
- **Code** (VARCHAR): The text voucher code entered at checkout (e.g., 'FRESH20').
- **DiscountType** (VARCHAR): The type of discount applied (e.g., 'Percentage', 'FixedAmount').
- **Value** (DECIMAL): The value or percentage of the discount.
- **StartDate** (DATE): When the promotion becomes active.
- **EndDate** (DATE): When the promotion expires.**Normalization Justification (Why It Is in 3NF)**

We structured our database to completely satisfy **Third Normal Form (3NF)** rules, ensuring our application stays clean, scalable, and free of annoying update bugs:

- **First Normal Form (1NF):** Every table column holds atomic, indivisible values. We avoided awkward multi-value text fields by separating complex data like order items and reviews into their own dedicated relational tables.
- **Second Normal Form (2NF):** All tables are already in 1NF, and every non-key attribute is fully dependent on the primary key. For example, a product's name and price depend strictly on its ProductID, preventing any partial dependencies.
- **Third Normal Form (3NF):** We eliminated all transitive dependencies. Non-key columns depend *only* on the primary key, meaning you won't find redundant data floating around where it doesn't belong. For instance, category names live exclusively in the Categories table rather than being repeatedly hardcoded into the Products table, making future updates quick and painless.

Chapter 5 — SQL Queries & Implementation

1. Low-Stock Inventory Alert Query

Purpose: Allows a Shop Admin to instantly monitor stock levels across their inventory and pinpoint items that have hit or fallen below their safety reorder threshold.

1) and isolates rows where the physical stock quantity is less than or equal to the designated low stock threshold, driving the dashboard alerts.

2. Shop Sales & Net Earnings Calculation

Purpose: Gives Shop Admins a clear breakdown of their financial performance over a selected period, separating gross revenue from net earnings after platform commission deductions.

3. Super Admin Global Commission & Revenue Report

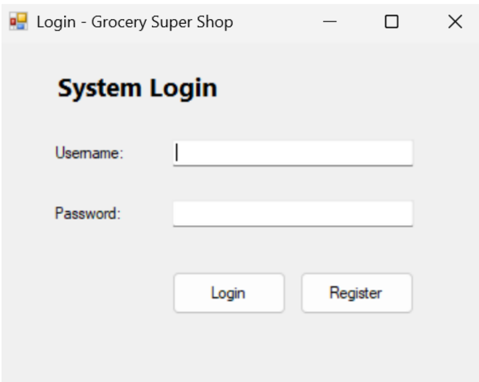
Purpose: Enables the Super Admin to audit platform-wide financial statistics, compiling total orders, aggregate revenue, and platform commission cuts across every active vendor store.

4. Customer Product Search & Category Filter

Purpose: Powers the customer storefront browsing experience by allowing users to narrow down product catalogs by specific departments and keyword searches.

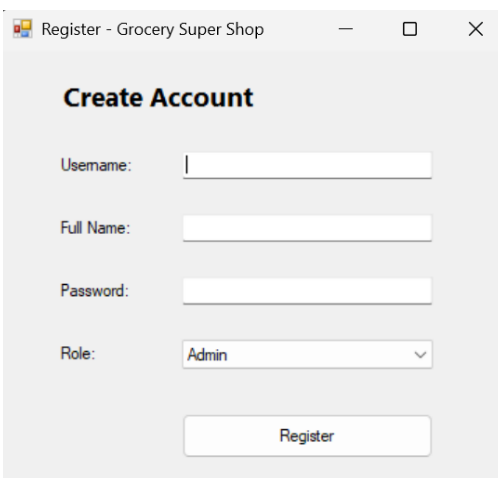
Chapter 6 — User Interface Design

Login Form:



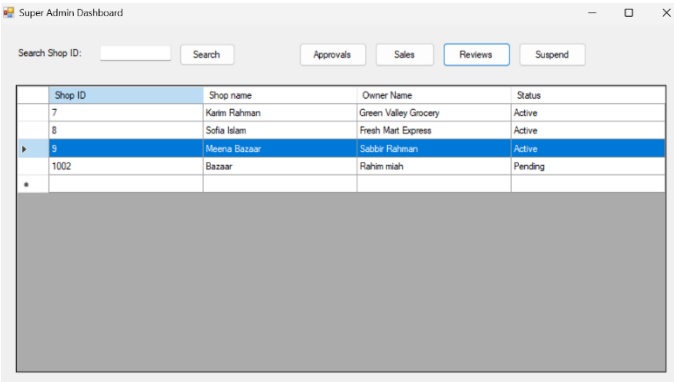
Provides input fields for a username and password alongside login and register buttons to authenticate system users.

Register Form:



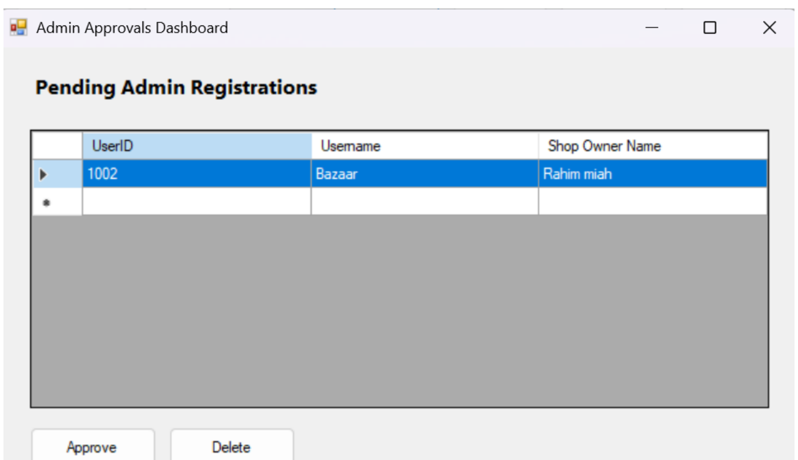
Allows new users to create an account by providing their username, full name, password, and selecting their role type.

Super Admin Dashboard:



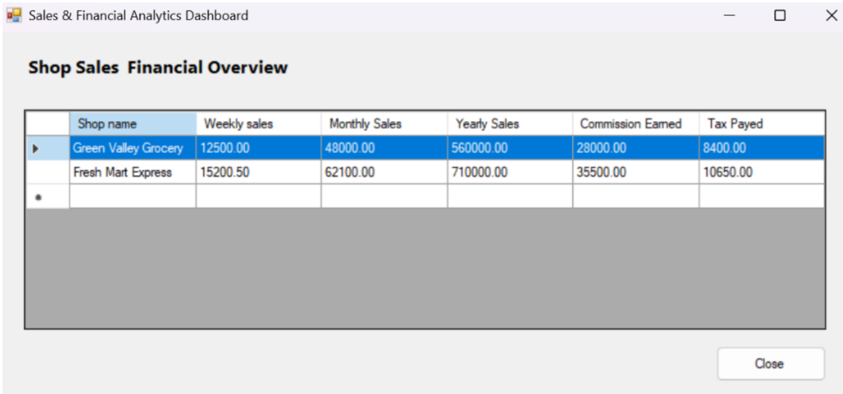
Provides top-level oversight of registered shops with tools to search, approve, review, or suspend vendor accounts.

Admin Approvals Dashboard:



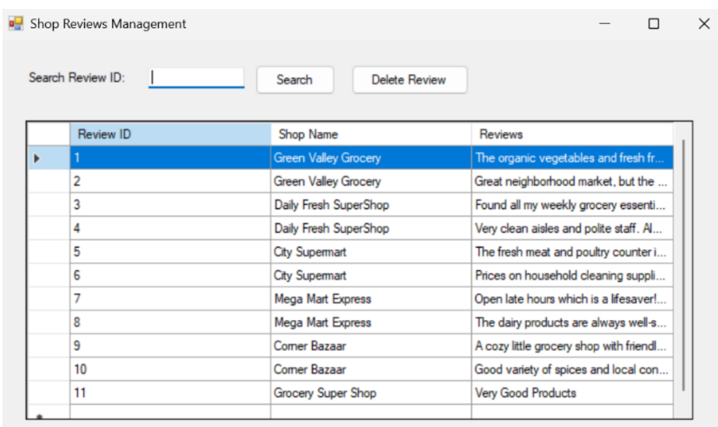
Lists pending store admin registrations and provides controls to either approve or delete account requests.

Sales Dashboard:



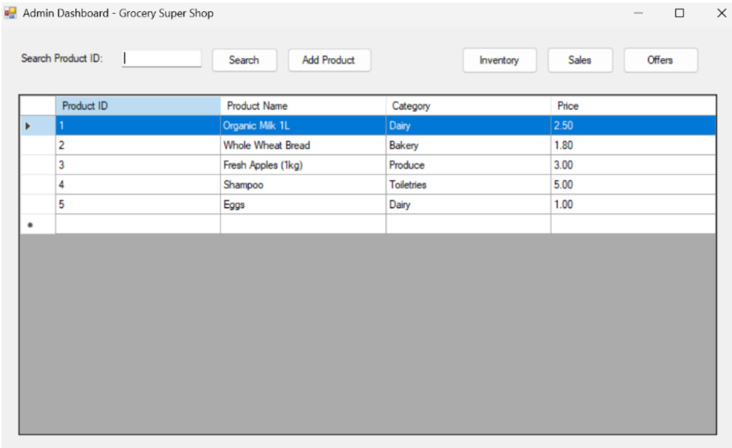
Displays financial performance metrics including weekly, monthly, and yearly sales, as well as commission earned and tax paid for each registered shop.

Manage Reviews Dashboard:



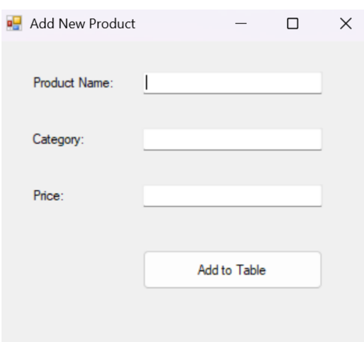
Lists customer reviews associated with various shop names and provides tools to search or delete specific feedback entries.

Admin Dashboard:



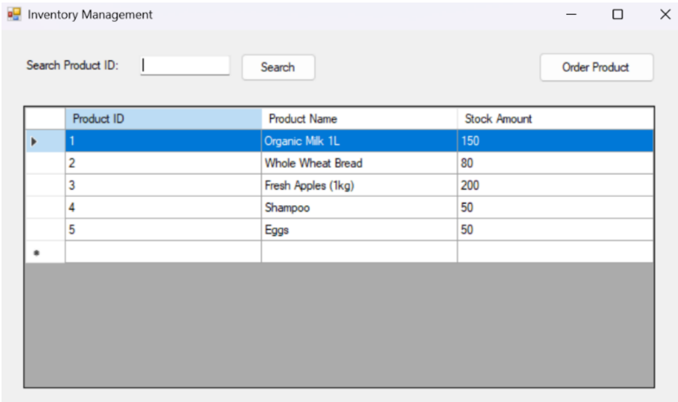
Serves as the main store management hub where administrators can search inventory, view product lists, and access product addition, inventory, sales, and promotional offer modules.

Add new Product:



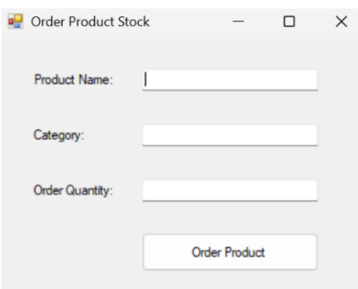
Enables administrators to input a new product's name, category, and price before adding it to the system table.

Inventory Dashboard:



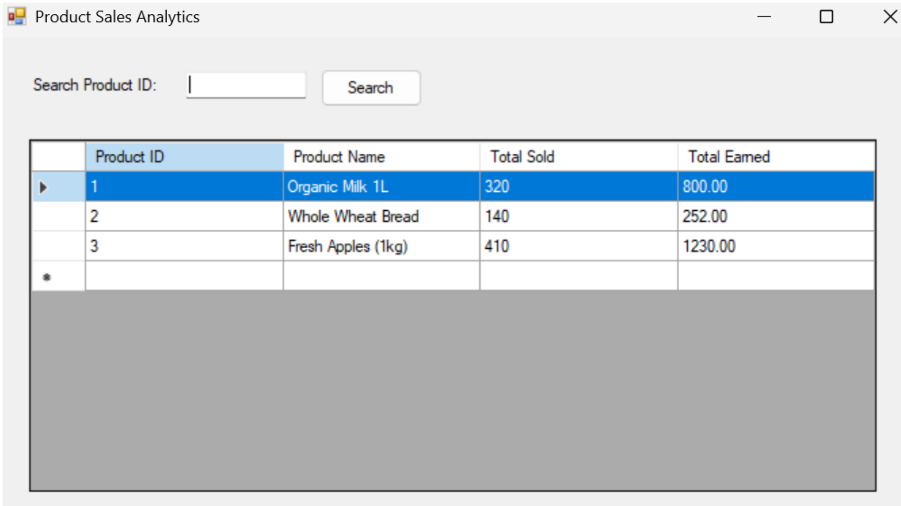
Shows current stock amounts for each product and lets administrators search inventory or trigger product stock orders.

Order Product:



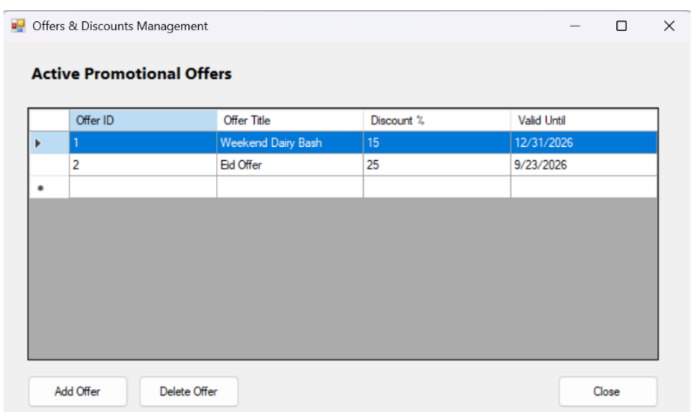
Enables administrators to input product names, categories, and order quantities to restock inventory items.

Shop Sales Dashboard:



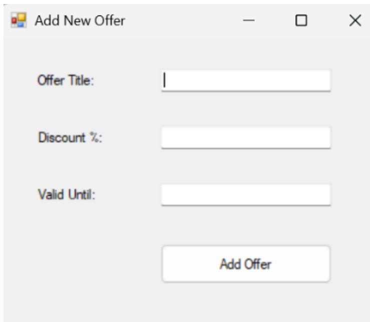
Displays product-specific performance metrics including total units sold and total revenue earned for individual items.

Offers Dashboard:



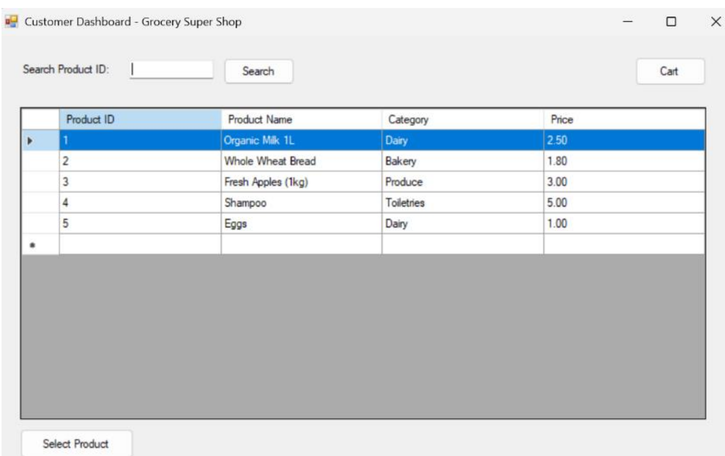
Displays active promotional offers with discount percentages and expiration dates, offering options to add or delete discounts.

Add new Offer:



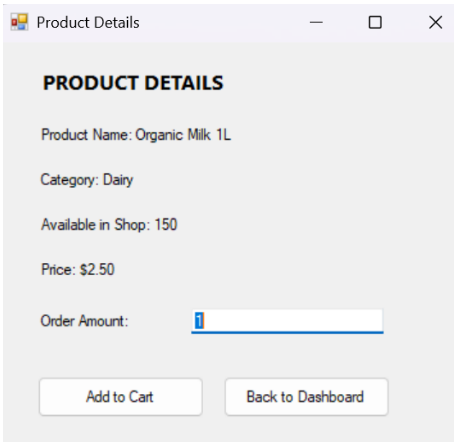
Lets administrators input promotional offer titles, discount percentages, and validity dates to create new discounts.

Customer Dashboard:



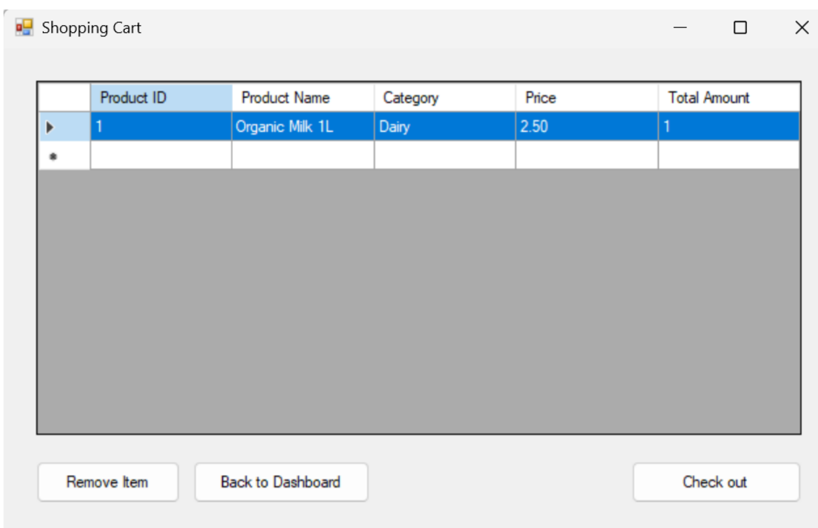
Displays available products with their IDs, names, categories, and prices, and includes a product search bar and cart navigation.

Product Details Dashboard:



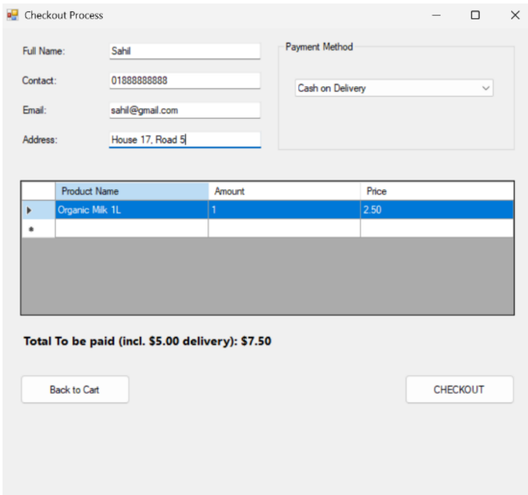
Displays specific item pricing, stock levels, and category details to let customers customize order quantities before adding items to their cart.

Shopping Cart Dashboard:



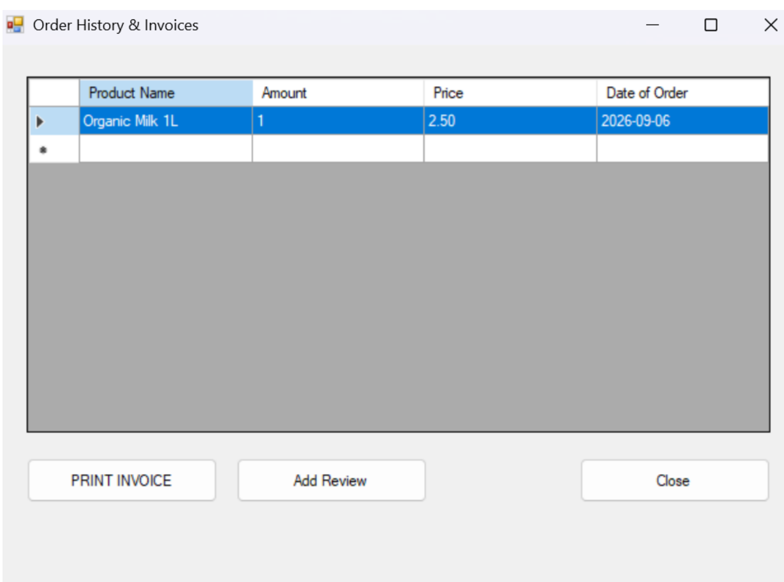
Lists items selected for purchase with their respective categories, prices, and quantities, accompanied by options to remove items or proceed to checkout.

Checkout Dashboard:



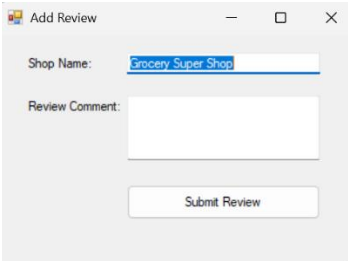
Collects customer delivery details and payment methods while providing an itemized order summary and total cost calculation.

Order History and Invoice Dashboard:



Shows past customer orders including item amounts, prices, and order dates, with options to print invoices, add reviews, or close the window.

Add Review:



Allows customers to submit textual feedback and comments associated with a specific shop name.

Chapter 7 — Work Distribution

Name	ID	Contribution
SAHIL SUBAIR	25-62082-2	Customer Whole Dashboard
NAFIM ISLAM NABID	24-58032-2	Database Connection
ESRAT JAHAN AKHI	23-55468-3	Admin Whole Dashboard
REDWANUL ALLAM SEAM	23-51158-1	Super Admin Whole Dashboard

Chapter 8 — Conclusion & Future Work

Building the **Grocery Super Shop** platform has been a rewarding journey in designing a robust, multi-vendor desktop application using C# WinForms and standard ADO.NET patterns. By structuring our database schema up to 3NF and securing dynamic queries with parameterized commands, we created an efficient system that seamlessly connects administrators, shop owners, and customers.

Looking ahead, our next milestone is transitioning this desktop architecture into a cloud-connected ecosystem. We plan to introduce real-time order tracking notifications, dynamic product recommendation logic based on buying habits, and an automated multi-vendor settlement system.

As the application scales, we anticipate key technical hurdles, particularly around managing concurrent database updates during peak shopping hours, keeping inventory synchronized seamlessly across independent vendors, and ensuring top-tier security for automated payment integrations. Tackling these upcoming challenges will be essential as we evolve this project into a enterprise-ready retail solution.